CSI370 Computer Architecture

Software Project: Verte Compiler - Assembly Refactor

Andy Zheng
Champlain College
andy.zheng@mymail.champlain.edu

November 28, 2025

1 Project Overview

1.1 What: Project Description

This project aims to refactor the existing code generation backend of the `vertec` compiler. `vertec` is a compiler built for the `Verte` programming language which currently utilizes `LLVM` for machine code generation. With the proposed project, the `LLVM` backend will be replaced with a custom, hand-written `x86-64 Assembly` backend. This new backend will directly translate the `AST` (abstract syntax tree) created by the current frontend into `x86-64 Assembly` instructions.

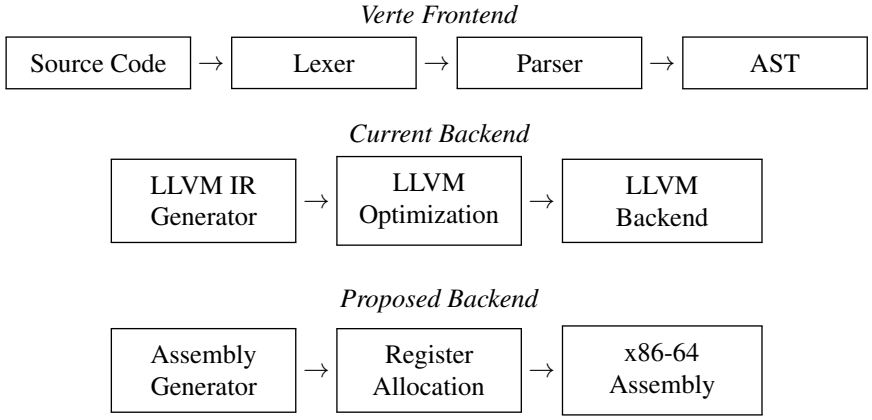

Figure 1: Verte Compiler architecture, showing existing and proposed backends.

1.2 Why: Motivation

The motivation behind this project is for both educational and technical reasons. I believe that building a custom assembly backend will provide invaluable insights into low-level code generation, and compilation. And, specifically in the context of this class, doing this project will deepen my understanding of `x86-64 Assembly` instructions, calling conventions, and register/instruction selection. Besides the educational reasons, I also feel it would be a good idea to explore alternatives to `LLVM` due to it's complexity and steep learning curve. Building a custom backend will thus, allow for a more tailored and simplified code generation process that is easier to understand and maintain.

1.3 How: Approach and Methodology

The implementation of this refactor will be carried out in several phases over the rest of this course. It will start with an analysis of the existing `LLVM` backend integration to understand best how to replace it. Following that, designing the architecture for the assembly generation will be done. Whilst development is ongoing, test cases will be created to validate correctness, once assembly generation is functional and correctness is ensured, the refactorization is complete.

Table 1: Key source files and their roles for the refactor.

File	Purpose	Implementation
`visitors/asm_visitor.[hpp/cpp]`	AST traversal	Visitor pattern for nodes
`codegen/instruction_selector.[hpp/cpp]`	Instruction mapping	x86-64 instruction selection
`codegen/register_allocator.[hpp/cpp]`	Register management	Register allocation
`codegen/asm_emitter.[hpp/cpp]`	Code emission	NASM text generation
`codegen/calling_convention.[hpp/cpp]`	ABI compliance	System V calling convention
`codegen/compiler.[hpp/cpp]*`	Integration	Modify to use assembly backend

*Indicates modification of existing file; all others are new files.

1.4 Challenges: Anticipated Difficulties

The project presents several technical and integration challenges that will need to be addressed. Specifically, managing register allocation and stack frames in `x86-64 Assembly` can be complex. Proper handling of function calls and adhering to calling conventions will also be critical to ensure generated code correctness. Thus careful planning and testing will be essential to overcome these challenges.

1.4.1 Solutions

To mitigate the anticipated challenges, I plan to take a systematic approach. This includes thorough research into `x86-64 Assembly` programming and calling conventions. Additionally, I will implement the backend

incrementally, starting with simple features and gradually adding complexity. Testing will be conducted at each stage to ensure correctness and to catch issues early. I will also leverage existing resources and tools, such as documentation, debuggers, and tutorials, to aid in the development process.

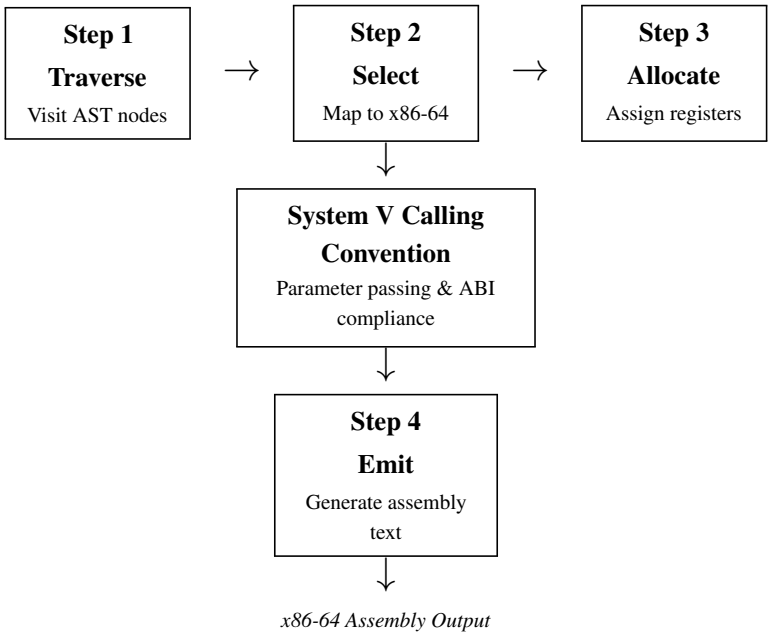

Figure 2: Incremental workflow from transforming AST into x86-64 Assembly code with System V ABI compliance.